САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа №2

Выполнил:

Корольков Артем Алексеевич

К3340

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Современные веб-приложения требуют чётко спроектированной серверной части, обеспечивающей обработку пользовательских запросов, хранение данных и взаимодействие с клиентскими интерфейсами. Одним из ключевых этапов разработки backend-системы является формализация требований и проектирование API.

Целью данной лабораторной работы является определение функциональных и нефункциональных требований к backend-приложению бронирования ресторанов, выбор архитектурного подхода к построению API, а также разработка спецификации API в формате OpenAPI с учётом ранее спроектированной базы данных.

Ход работы

1. Функциональные и нефункциональные требования

Функциональные требования определяют поведение системы и описывают, какие действия пользователь может выполнять с её помощью. В рамках разрабатываемого приложения система должна обеспечивать полный цикл взаимодействия пользователя с сервисом бронирования ресторанов.

Система должна обеспечивать следующий функционал:

Работа с пользователями

- Регистрация пользователя
- Аутентификация (вход)
- Просмотр и редактирование профиля
- Просмотр истории бронирований

Работа с ресторанами

- Получение списка ресторанов
- Фильтрация по:
    - типу кухни

    - городу
    - ценовому диапазону
- Просмотр подробной информации о ресторане

Меню

- Получение списка блюд ресторана

Отзывы

- Добавление отзыва
- Просмотр отзывов о ресторане

Бронирование

- Создание бронирования
- Просмотр списка бронирований пользователя
- Отмена бронирования

Прежде всего, система должна предоставлять возможность регистрации новых пользователей с последующей аутентификацией. После успешного входа пользователь получает доступ к личному кабинету, в котором он может просматривать и управлять своими бронированиями.

Одной из ключевых функций приложения является поиск ресторанов. Пользователь должен иметь возможность получать список доступных заведений, а также применять фильтры по различным параметрам, таким как тип кухни, географическое расположение и ценовой диапазон. Кроме того, система должна предоставлять подробную информацию о каждом ресторане, включая описание, меню, фотографии и отзывы других пользователей.

Функциональность бронирования включает создание нового бронирования с указанием даты, времени, количества гостей и выбранного столика. Также пользователь должен иметь возможность просматривать историю своих бронирований и отменять ранее созданные записи.

Дополнительно система должна поддерживать работу с отзывами, позволяя пользователям оставлять оценки и комментарии к ресторанам, а также просматривать отзывы других пользователей.

Нефункциональные требования определяют характеристики системы, не связанные напрямую с её функциональностью, но влияющие на качество работы. Важнейшим требованием является производительность системы: сервер должен обрабатывать значительное количество запросов без заметных задержек. Также система должна быть масштабируемой, что позволит увеличивать её производительность за счёт добавления ресурсов.

Нефункциональные требования:

- **Производительность:** API должно обрабатывать не менее 100 запросов в секунду
- **Масштабируемость:** возможность горизонтального масштабирования
- **Безопасность:**
  - хранение паролей в хэшированном виде
  - использование JWT-токенов
- **Надёжность:** обработка ошибок и валидация данных
- **Расширяемость:** возможность добавления новых функций без изменения архитектуры
- **Документированность:** наличие OpenAPI-спецификации

Особое внимание уделяется безопасности. Персональные данные пользователей должны храниться с использованием современных методов защиты, включая хэширование паролей и применение механизмов аутентификации на основе токенов. Надёжность системы обеспечивается корректной обработкой ошибок и предотвращением некорректных состояний данных.

Кроме того, система должна быть расширяемой, что позволит в дальнейшем добавлять новые функции без необходимости значительной переработки архитектуры. Немаловажным требованием является наличие документации API, обеспечивающей удобство интеграции и сопровождения системы.

2. Выбор архитектурного подхода к реализации API

При проектировании серверной части приложения был рассмотрен выбор между подходами REST API и Backend For Frontend (BFF). В результате анализа был выбран подход REST API.

Данный выбор обусловлен тем, что REST является широко распространённым архитектурным стилем, основанным на использовании стандартных HTTP-методов и ресурсов. Он обеспечивает простоту разработки, универсальность и совместимость с различными клиентскими приложениями.

REST API позволяет чётко структурировать взаимодействие между клиентом и сервером, используя единообразные интерфейсы. Кроме того, данный подход хорошо интегрируется с инструментами документирования, такими как OpenAPI, что упрощает процесс разработки и тестирования.

Таким образом, использование REST API является оптимальным решением для реализации backend-приложения бронирования ресторанов.

3. Проектирование API и OpenAPI-спецификация

На основе ранее разработанной ERD-диаграммы была спроектирована структура API, отражающая основные сущности системы и операции над ними. Для формального описания API была использована спецификация OpenAPI, позволяющая стандартизировать взаимодействие между клиентом и сервером.

API включает в себя набор эндпоинтов, сгруппированных по функциональным областям: аутентификация, работа с ресторанами, бронированиями и отзывами.

Ниже представлена разработанная OpenAPI-спецификация:

```
openapi: 3.0.0
info:
  title: Restaurant Booking API
  version: 1.0.0

paths:

  /auth/register:
    post:
```

```
      summary: Регистрация пользователя
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email, password, name]
              properties:
                email:
                  type: string
                  example: user@mail.com
                password:
                  type: string
                  example: password123
                name:
                  type: string
                  example: Ivan Ivanov
      responses:
        201:
          description: Пользователь успешно создан
          content:
            application/json:
              schema:
                type: object
                properties:
                  id:
                    type: integer
                  email:
                    type: string
        400:
          description: Ошибка валидации

  /auth/login:
    post:
      summary: Аутентификация пользователя
```

```yaml
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [email, password]
              properties:
                email:
                  type: string
                password:
                  type: string
      responses:
        200:
          description: Успешный вход
          content:
            application/json:
              schema:
                type: object
                properties:
                  token:
                    type: string
        401:
          description: Неверные учетные данные

  /restaurants:
    get:
      summary: Получение списка ресторанов
      parameters:
        - name: city
          in: query
          schema:
            type: string
        - name: cuisine
          in: query
          schema:
            type: string
```

```yaml
        - name: price_range
          in: query
          schema:
            type: string
      responses:
        200:
          description: Список ресторанов
          content:
            application/json:
              schema:
                type: array
                items:
                  type: object

  /restaurants/{id}:
    get:
      summary: Получение информации о ресторане
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        200:
          description: Данные ресторана
        404:
          description: Ресторан не найден

  /reservations:
    post:
      summary: Создание бронирования
      requestBody:
        content:
          application/json:
            schema:
```

```yaml
              type: object
              required: [table_id, reservation_datetime,
guests_count]
              properties:
                table_id:
                  type: integer
                reservation_datetime:
                  type: string
                  example: "2026-04-01T18:00:00"
                guests_count:
                  type: integer
      responses:
        201:
          description: Бронирование создано
        400:
          description: Ошибка данных

    get:
      summary: Получение списка бронирований пользователя
      responses:
        200:
          description: Список бронирований

  /reservations/{id}:
    delete:
      summary: Отмена бронирования
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        200:
          description: Бронирование отменено
        404:
```

```
          description: Не найдено
```

4. Описание запросов, ответов и обработки ошибок

Каждый эндпоинт API принимает и возвращает данные в формате JSON. Тело запроса содержит необходимые параметры для выполнения операции, например, при регистрации пользователя передаются адрес электронной почты, пароль и имя.

Ответы сервера формируются в зависимости от результата выполнения операции. В случае успешного выполнения возвращается код состояния HTTP (например, 200 или 201) и данные, соответствующие запросу. Например, при успешной аутентификации пользователь получает токен доступа, который используется для последующих запросов.

В случае возникновения ошибок сервер возвращает соответствующий HTTP-код и описание ошибки. Для унификации обработки ошибок используется стандартная структура ответа, содержащая тип ошибки и сообщение.

Пример ответа при ошибке:

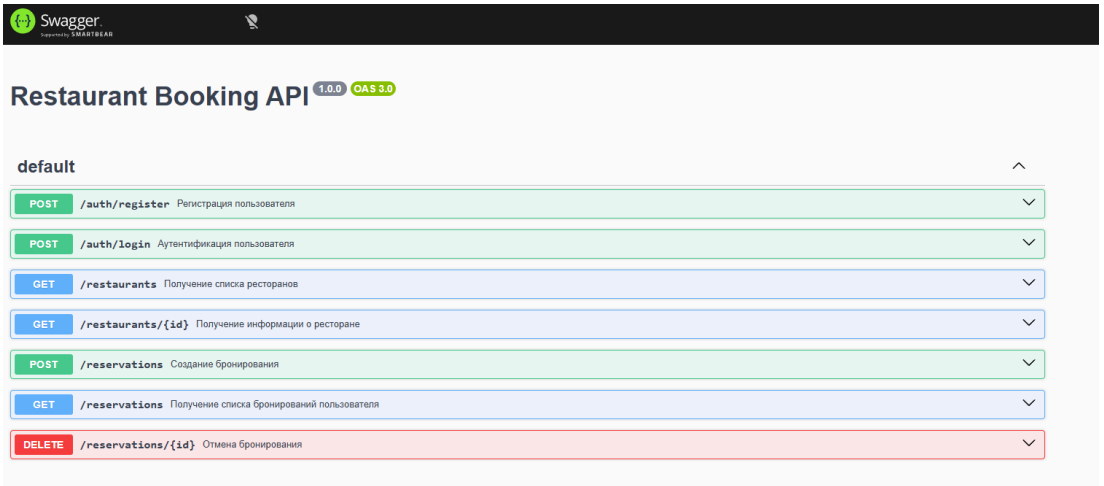
```
{
 "error": "ValidationError",
 "message": "Invalid input data"
}
```

Система должна обрабатывать различные типы ошибок, включая ошибки валидации входных данных, ошибки аутентификации, отсутствие запрашиваемого ресурса и внутренние ошибки сервера. Это позволяет обеспечить устойчивость системы и улучшить пользовательский опыт.

5. Реализация и тестирование API с использованием Swagger

В процессе выполнения лабораторной работы разработанная OpenAPI-спецификация была интегрирована в backend-приложение. Для визуализации и тестирования API использовался инструмент Swagger UI.

После запуска серверного приложения в браузере стала доступна интерактивная страница документации API по адресу http://localhost:3000/api-docs. На данной странице отображаются все реализованные эндпоинты, сгруппированные по функциональному назначению, включая методы аутентификации, работы с ресторанами и бронированиями.



Каждый эндпоинт содержит подробное описание, включая HTTP-метод, путь запроса, структуру входных данных и возможные ответы сервера. Swagger UI также предоставляет возможность отправки тестовых запросов непосредственно из браузера с использованием функции «Try it out», что позволяет проверить корректность работы API без использования сторонних инструментов.

Таким образом, использование Swagger UI значительно упрощает процесс разработки, тестирования и документирования API, а также обеспечивает наглядное представление структуры взаимодействия клиента с сервером.

Вывод

В ходе выполнения лабораторной работы были подробно рассмотрены требования к backend-приложению бронирования ресторанов, включая как функциональные, так и нефункциональные аспекты. На основе проведённого анализа был выбран архитектурный стиль REST API, обеспечивающий удобство разработки и масштабируемость системы.

Была разработана OpenAPI-спецификация, описывающая основные эндпоинты и структуру взаимодействия клиента с сервером. Особое внимание было уделено описанию форматов запросов и ответов, а также обработке возможных ошибок.

Разработанное API соответствует ранее спроектированной структуре базы данных и может быть использовано в качестве основы для реализации серверной части приложения. Полученные результаты демонстрируют практические навыки проектирования backend-систем и подготовки технической документации.

Таким образом, все поставленные задачи лабораторной работы были успешно выполнены.